

Symmetry and Scale Separation in Convolutional Neural Networks

Task 2 Report

Simachew Alamneh

September 2026

Abstract

In Task 1 we built a CNN and a feed-forward network (FFNN) from scratch, using only NumPy, and compared them on Fashion-MNIST. On centred images both networks reach about 85% test accuracy, yet the CNN uses four times fewer parameters, overfits less, and keeps more of its accuracy under small shifts and zooms — while being slower to train and heavier in memory. This report explains *why*, with the mathematics of two properties image data has and ordinary dense networks cannot exploit: **symmetry** (a shoe is still a shoe wherever it appears in the picture) and **scale separation** (an image is built from edges, which form parts, which form objects, so a network can look locally first and combine information at larger scales later). Every theorem below is proved in full, step by step, and is paired with a numerical check computed from our own trained models, so that the abstract claim, its derivation, and a concrete number sit next to each other throughout.

Contents

1	Notation	2
2	Introduction	2
3	Symmetry: basic definitions	5
4	Convolution is translation equivariant	5
5	Pooling: from equivariance to invariance	7
6	Where the symmetry stops: rotation and zoom	9
7	Scale separation	12
7.1	The Fourier transform, and what it cannot tell you	12
7.2	The wavelet transform: localising both position and scale	13
7.3	From wavelets to CNNs: a learned, cascaded multi-resolution pyramid	14
7.4	Receptive field: the concrete numbers	15
7.5	Cost, approximation and stability	16
8	Link with the Task 1 results	17
9	Limitations	17

1 Notation

Before the mathematics, this section collects every symbol used later, so that no formula needs to be reread twice for what a letter means.

Symbol	Meaning
u, v	A position (pixel coordinate) on the 2-D pixel grid, $u = (u_1, u_2) \in \mathbb{Z}^2$.
$x(u)$	The value (intensity) of an image x at pixel u ; the image is treated as a function $x : \Omega \rightarrow \mathbb{R}$ on the grid Ω .
$w(v)$	A convolution kernel (filter), a small function defined on a local neighbourhood, e.g. 3×3 .
s	A shift vector, $s = (s_1, s_2) \in \mathbb{Z}^2$, used to translate an image.
T_s	The translation (shift) operator: $(T_s x)(u) = x(u - s)$.
$\star$	Convolution (in this report, technically cross-correlation, which is what CNN libraries implement): $(x \star w)(u) = \sum_v x(u + v) w(v)$.
R, R_θ	A rotation of the plane; R_θ denotes rotation by angle θ . Acts on images by $(Rx)(u) = x(R^{-1}u)$.
D_a	The zoom (dilation) operator by factor a : $(D_a x)(u) = x(u/a)$.
δ_v	The impulse (indicator) image: $\delta_v(u) = 1$ if $u = v$, else 0.
P	A pooling (coarsening) operator, e.g. $(Px)(u') = \max_{v \in \{0,1\}^2} x(2u' + v)$ for 2×2 max pooling with stride 2.
r_l, j_l	The receptive-field size and the “jump” (spacing between neighbouring units) after layer l .
k_l, s_l	The kernel size and stride of layer l .
ω	Spatial frequency, the variable of the Fourier transform.
$\hat{x}(\omega)$	The Fourier transform of x : $\hat{x}(\omega) = \int x(u) e^{-i\omega u} du$.
i	The imaginary unit, $i = \sqrt{-1}$.
$\psi(u)$	A mother wavelet: a small, localised, oscillating function with zero average, used to build a multi-resolution decomposition.
$\psi_j(u)$	The mother wavelet dilated to scale 2^j : $\psi_j(u) = 2^{-j} \psi(u/2^j)$.
$(x \star \psi_j)(u)$	The wavelet coefficient of x at scale j and position u — itself an ordinary convolution, with kernel ψ_j .
j	The scale, or octave, index of a wavelet or of a pooling stage; larger j means coarser.
■	End of proof.

2 Introduction

A Convolutional Neural Network (CNN) is a type of artificial neural network designed specifically for data that has a grid-like spatial structure, most commonly images, where each pixel’s meaning depends on the pixels around it. Unlike a plain feed-forward neural network (FFNN), which treats its input as an unstructured list of numbers and connects every input unit to every unit of the next layer, a CNN is built from layers that look at small local neighbourhoods of the input at a time and reuse the same small set of weights at every position of the image. This single design decision

— looking locally and reusing weights — is the source of almost everything a CNN is good at, and it is also the reason this report exists: it is a mathematical property (a symmetry) rather than a heuristic, and it can be proved, not just observed.

A CNN is normally built by stacking a small number of layer types, each doing one simple job. A **convolution layer** slides a small learnable filter, or **kernel** (typically 3×3 or 5×5 pixels), across the image and, at every position, computes a weighted sum of the pixels currently under the kernel. The same kernel weights are reused at every position — this is **weight sharing** — so a single kernel that has learned to detect, say, a vertical edge, will detect vertical edges anywhere in the image, not only where it happened to see one during training. Because the kernel only looks at a small neighbourhood at a time rather than the whole image at once, the layer is said to have **local connectivity**: an output pixel depends only on a handful of nearby input pixels, not on the entire image, in contrast to a dense (fully-connected) layer where every output depends on every input. Sliding the kernel produces a **feature map**, and a convolution layer normally learns several kernels in parallel, each producing its own feature map (its own **channel**) that responds to a different pattern (edges at different orientations, colours, simple textures).

Three further, closely-related settings control exactly how a convolution layer behaves. The **stride** is the step size the kernel moves by between two positions; a stride of 1 slides the kernel one pixel at a time and produces an output roughly the same size as the input, while a stride of 2 skips every other position and halves the output resolution, which is a cheap way to shrink the feature map as it also does some of the job a pooling layer does. **Padding** is the (usually zero-valued) border added around the input before the kernel is applied; without padding (*valid* convolution) the output shrinks by $k - 1$ pixels in each dimension for a $k \times k$ kernel, and pixels near the border are seen by fewer kernel positions than pixels in the middle, so padding (*same* convolution) is used when the output should keep the input’s size and border pixels should be treated fairly. A **pooling layer** (most commonly 2×2 max pooling with stride 2) is a second, non-learned way of shrinking the feature map: it replaces each small block of the feature map by a single summary number, usually the maximum, which both reduces the amount of computation needed by later layers and makes the representation insensitive to small shifts of a feature within its pooling window, as this report proves formally in Section 5.

A typical CNN alternates a handful of convolution (plus nonlinearity, e.g. ReLU) and pooling layers, and then **flattens** the last, small, many-channel feature map into a single long vector, which is finally passed through one or two ordinary dense (fully-connected) layers ending in as many output units as there are classes. Going from input to output, the image therefore passes through progressively smaller, progressively deeper (more-channel) representations: a $28 \times 28 \times 1$ image might become $26 \times 26 \times 8$ after the first convolution, $13 \times 13 \times 8$ after pooling, $11 \times 11 \times 16$ after a second convolution, $5 \times 5 \times 16$ after a second pooling, and finally a 400-number vector that two dense layers turn into 10 class scores. An ordinary FFNN given the same image skips all of this: the 28×28 grid is flattened into a 784-number vector on the very first step, so the network never has the chance to see which pixels were spatial neighbours of which — a pixel at the top-left corner and a pixel at the bottom-right corner become just two entries of a list, exactly as unrelated to each other, as far as the first dense layer is concerned, as two entries that actually were neighbours. Figure 1 shows this input-to-output path through a CNN.

This is the basic motivation for using a CNN on image data in the first place. Images have two properties that a plain FFNN cannot exploit but a CNN is built around. The first is **symmetry**: an object’s identity does not depend on where in the image it appears, so a good feature detector, once learned, should be reusable at every position rather than relearned separately for every possible location — this is exactly what weight sharing provides, and Section 3 onward makes this precise as *translation equivariance*. The second is **scale separation**: an image is compositional, built

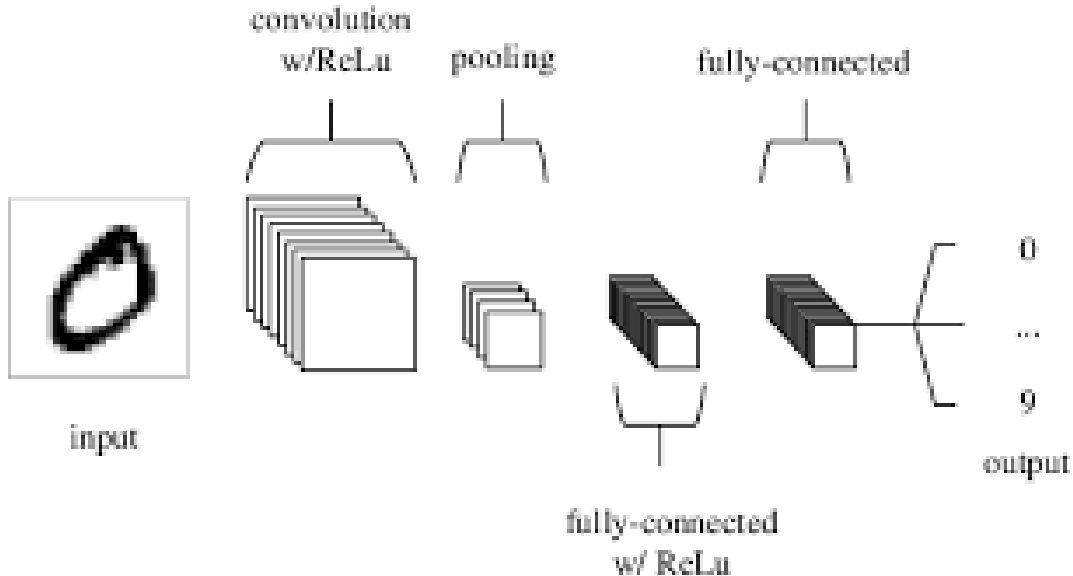


Figure 1: A typical CNN architecture, from input image to output class scores: alternating convolution (with ReLU) and pooling layers extract and coarsen local features, and fully-connected layers at the end combine them into a classification decision.

from small parts (edges) that combine into larger parts (corners, textures) that combine into whole objects, so a network that first looks at small local neighbourhoods and only gradually widens its field of view, layer by layer, can build a hierarchical description of the image far more efficiently — in far fewer parameters — than a network that must consider the whole image at once from its very first layer. Because an FFNN has neither of these properties built in, it must, in effect, relearn the same edge detector separately at every possible pixel position and must relate every pixel to every other pixel directly rather than hierarchically; the parameter counts above already show the cost of this (a dense layer with the same input and output size as our first convolution layer needs 4,239,872 weights, against 80 for the convolution).

Our own Task 1 experiment illustrates the consequence directly. On Fashion-MNIST images as they normally appear (centred, not transformed), both networks reach about the same test accuracy (FFNN 85.6%, CNN 84.8%). The CNN, however, does so with four times fewer parameters (27,562 against 109,386), a smaller gap between training and test accuracy (5.7 against 7.2 points), and it keeps substantially more of its accuracy when the test images are shifted or zoomed a little; it is, in our NumPy implementation, also considerably slower to train (331 s against 38 s), since weight sharing reduces the number of *parameters* but not the number of *operations*. The remainder of this report explains this pattern of results — and its limits — with mathematics: what exactly a CNN is symmetric to, why it is symmetric to it, and what happens, both mathematically and in our own numbers, when the test data is transformed in ways the CNN’s symmetry does not cover. Every theorem below is proved in full and followed by a short numerical check, computed with a small NumPy script (`symmetry_checks.py`) against our own trained Task 1 models, so that the abstract claim and the concrete number sit next to each other throughout.

3 Symmetry: basic definitions

We treat an image as a function on the pixel grid: $x(u)$ is the pixel value at position $u = (\text{row}, \text{column})$. Shifting the image by a vector s gives a new image

$$(T_s x)(u) = x(u - s). \quad (1)$$

Read this as: the value of the shifted image at position u is whatever the original image had at the position you would need to shift backward, $u - s$, to reach u .

A function f is **invariant** to shifts if $f(T_s x) = f(x)$: the output does not change when the input moves. A layer F is **equivariant** if $F(T_s x) = T_s F(x)$: if the input moves, the output moves in the same way. We want the first layers of a network to be equivariant (they need to know *where* a feature is) and the final decision to be invariant (the class does not depend on position).

4 Convolution is translation equivariant

The convolution used in CNNs (strictly a cross-correlation) is

$$(x \star w)(u) = \sum_v x(u + v) w(v), \quad (2)$$

where w is a small kernel (for example 3×3) and the sum runs over the kernel positions. The same kernel is used at every position u .

Theorem 4.1. *For every kernel w and shift s , $(T_s x) \star w = T_s(x \star w)$.*

Proof. We show the two sides agree at every position u .

Step 1. Expand the left side using the definition of convolution, treating $T_s x$ as the image being convolved:

$$((T_s x) \star w)(u) = \sum_v (T_s x)(u + v) w(v).$$

Step 2. Expand the inner term $(T_s x)(u + v)$ using the shift definition, applied at the point $u + v$:

$$(T_s x)(u + v) = x((u + v) - s) = x(u + v - s).$$

Substituting back,

$$((T_s x) \star w)(u) = \sum_v x(u + v - s) w(v). \quad (\text{L})$$

Step 3. Expand the right side. By the shift definition applied to the whole function $x \star w$, evaluated at $u - s$:

$$(T_s(x \star w))(u) = (x \star w)(u - s).$$

Step 4. Expand $(x \star w)(u - s)$ using the definition of convolution, with $u - s$ as the centre point:

$$(x \star w)(u - s) = \sum_v x((u - s) + v) w(v) = \sum_v x(u - s + v) w(v). \quad (\text{R})$$

Step 5. Compare (L) and (R). The arguments inside $x(\cdot)$ are $u + v - s$ and $u - s + v$: the same number, since addition of integers commutes. So (L) and (R) are term-by-term identical, for every u . Hence $(T_s x) \star w = T_s(x \star w)$. $\square$

Why this works, in one sentence. Convolution only ever reads the input through the combination $u + v - s$, never through u and s separately; because that combination does not care how its three terms are grouped, the $-s$ may equally well be pulled out of the sum and applied once, at the end, to the whole feature map.

A ReLU applied to each pixel also commutes with shifts (it acts identically and independently at every position), and a composition of equivariant layers is equivariant, so a whole stack of convolution and ReLU layers is equivariant.

Theorem 4.2 (Convolution is the only choice). *If a linear map L commutes with all shifts, i.e. $L(T_s x) = T_s(Lx)$ for every s , then it is a convolution: there exists a fixed image h such that $(Lx)(u) = \sum_v x(v) h(u - v)$.*

Proof. The idea is to probe L with the simplest possible image, a single lit pixel, and let linearity and shift-equivariance tell us what L does to every other image from that one probe.

Step 1. Let δ_v be the impulse image, 1 at pixel v and 0 elsewhere. Every impulse is a shifted copy of the impulse at the origin: $\delta_v = T_v \delta_0$. (Check: $(T_v \delta_0)(u) = \delta_0(u - v)$, which is 1 exactly when $u = v$, matching $\delta_v(u)$.)

Step 2. Any image is a sum of its own pixels, each treated as a scaled impulse:

$$x = \sum_v x(v) \delta_v.$$

(Check pointwise at a position u : the right side is $\sum_v x(v) \delta_v(u)$, and $\delta_v(u)$ is 0 for every v except $v = u$, so the sum collapses to $x(u)$, matching the left side.) This identity holds for every image, not just a special one.

Step 3. Apply L to both sides and use linearity, which allows L to pass through the sum and through the scalar weights $x(v)$:

$$Lx = L\left(\sum_v x(v) \delta_v\right) = \sum_v x(v) L(\delta_v).$$

This is the only step in the whole proof that uses linearity.

Step 4. Use Step 1, then shift-equivariance:

$$L(\delta_v) = L(T_v \delta_0) = T_v(L\delta_0).$$

The first equality is Step 1; the second is exactly the equivariance assumption, applied with δ_0 playing the role of the image being shifted.

Step 5. Define $h := L\delta_0$: a single, fixed image, not depending on x , obtained by probing L with one lit pixel at the origin. Substituting into Step 3 via Step 4:

$$Lx = \sum_v x(v) T_v h.$$

Step 6. Evaluate at a point u , using $(T_v h)(u) = h(u - v)$:

$$(Lx)(u) = \sum_v x(v) (T_v h)(u) = \sum_v x(v) h(u - v).$$

This is exactly the definition of convolution: $Lx = x \star h$. □

Why this works, in one sentence. A linear, shift-equivariant map cannot know where it is on the image — equivariance forbids it from treating position 5 differently from position 500 — so its entire behaviour is forced to be “do the same local thing everywhere”, and $h = L\delta_0$ is a complete recording of what that local thing is; knowing the reaction to one dot hands you, for free, the reaction to every other dot (shift the recording) and, by linearity, to every image (add the reactions up, since every image is a pile of dots).

Consequence for parameter count. As a matrix, an equivariant linear layer is circulant (each row is the previous row shifted), so it has only as many free numbers as the kernel has. A dense layer from 784 pixels to 784 outputs has 614,656 weights; a 3×3 convolution has 9. In our CNN the first layer has 80 parameters and produces $8 \times 26 \times 26 = 5,408$ outputs; a dense layer with the same input and output sizes would need $784 \times 5,408 = 4,239,872$ weights. This explains the four-times-smaller parameter count of the CNN.

Note on nonlinear layers. Theorem 4.2 needs linearity essentially: a nonlinear shift-equivariant map, such as ReLU or max-pooling, does not have to be a convolution, and indeed is not one — both are equivariant for a different, more elementary reason (they act identically and independently at every position, so shifting the input trivially shifts which position sees which value).

Numerical check. We took a Fashion-MNIST ankle boot (shrunk so it has empty margin) and a Sobel kernel. The largest difference between “shift then convolve” and “convolve then shift” was 0 for the valid convolution used in our CNN (Figure 2). The same test, repeated with a random dense layer instead of the convolution, gave a relative error of 1.405 — pure noise, confirming that an arbitrary linear layer does not have this property, while a convolution does. One limit: with valid convolution the equality is exact only while the object stays inside the image; parts that move over the border are cut off.

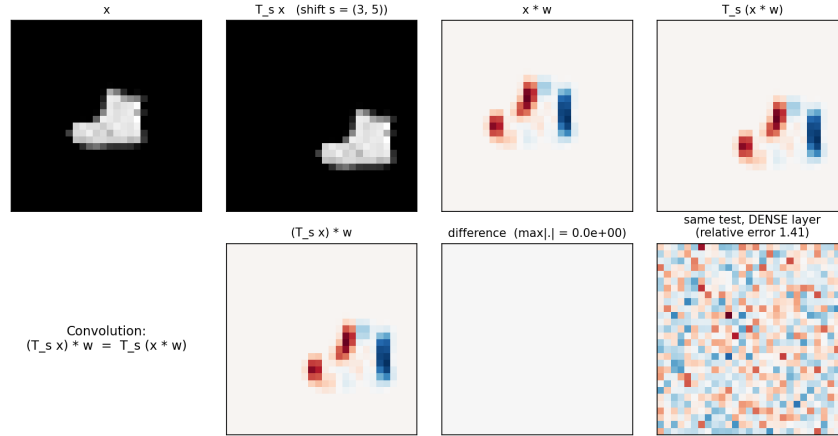


Figure 2: Shift then convolve equals convolve then shift (difference panel is zero). A dense layer does not have this property (bottom right).

5 Pooling: from equivariance to invariance

Our CNN uses 2×2 max pooling with stride 2:

$$(Px)(u') = \max_{v \in \{0,1\}^2} x(2u' + v). \quad (3)$$

To get the pooled pixel at coarse position u' , look at the 2×2 block of fine pixels starting at $2u'$, and take the maximum.

Theorem 5.1. *For stride 2, $P(T_{2s}x) = T_s P(x)$ for every integer shift s . Shifts that are multiples of the stride are handled exactly; the argument below does not extend to a shift by an odd number of pixels.*

Proof. We show the two sides agree at every coarse position u' .

Step 1. Expand the left side using the definition of pooling, applied to the shifted image $T_{2s}x$:

$$(P(T_{2s}x))(u') = \max_{v \in \{0,1\}^2} (T_{2s}x)(2u' + v).$$

Step 2. Expand the inner term using the shift definition:

$$(T_{2s}x)(2u' + v) = x((2u' + v) - 2s) = x(2u' - 2s + v) = x(2(u' - s) + v),$$

where the last equality just factors the 2 back out: $2u' - 2s = 2(u' - s)$.

Step 3. Substitute into Step 1:

$$(P(T_{2s}x))(u') = \max_{v \in \{0,1\}^2} x(2(u' - s) + v).$$

Step 4. Recognise the right side as the pooling formula evaluated at $u'' = u' - s$: comparing to $(Px)(u'') = \max_v x(2u'' + v)$,

$$\max_{v \in \{0,1\}^2} x(2(u' - s) + v) = (Px)(u' - s).$$

Step 5. Recognise $(Px)(u' - s)$ as a shift of the function Px , by the shift definition:

$$(Px)(u' - s) = (T_s(Px))(u').$$

Chaining Steps 1–5: $(P(T_{2s}x))(u') = (T_s(Px))(u')$ for every u' , so $P(T_{2s}x) = T_s P(x)$. $\square$

This is equivariance, not invariance. Invariance would mean $P(T_{2s}x) = P(x)$: the output stays exactly the same after a shift. What is actually proved is that the output also shifts, just by half as much and at half the resolution — position information survives, compressed. The proof needs the shift to be an even number of pixels: Step 2’s factoring $2u' - 2s = 2(u' - s)$ only works because the shift applied to x was $2s$, a multiple of the stride. For an odd shift, the 2×2 windows in Step 1 group the pixels differently, no such factoring is available, and in general no shift of the pooled output reproduces the result. This effect is called **aliasing**. We checked this directly: the identity holds with error 0 for even shifts, while for a 1-pixel shift the best coarse shift of the pooled output still leaves an error of 3.159.

Global pooling gives true invariance. If, instead of pooling locally, we sum over *every* remaining position, $g(x) = \sum_u x(u)$, then $g(T_s x) = g(x)$ exactly, for any shift s at all — once every position has been summed over, there is no position left to shift. So “equivariant layers followed by a global pooling” gives an exactly shift-invariant classifier. Our CNN does not do this: it flattens the last feature map (keeping position) rather than summing it, which is why the robustness it gains from pooling is only partial, as the next paragraph shows numerically.

Table 2 shows how much a representation changes when the test image is shifted diagonally by s pixels, using 8 random convolution-ReLU-maxpool filters. Values near 0 mean invariant, values near 1.4 mean the representation is completely different.

Table 2: Relative change of the representation after a shift of s pixels.

Shift (px)	Raw pixels	Flattened pooled maps	Global sum of pooled maps
0	0.000	0.000	0.0000
1	0.660	0.422	0.0436
2	0.981	0.772	0.0000
3	1.202	1.017	0.0436
4	1.333	1.214	0.0000
5	1.402	1.311	0.0436
6	1.414	1.388	0.0000

The *Global sum* column is exactly 0 at every even shift and a small, fixed 0.0436 at every odd shift — precisely the even/odd pattern the theorem predicts, made visible in a real measurement: even shifts are handled exactly, odd shifts leak a small, constant amount of aliasing error. The *Flattened pooled maps* column, which is what our CNN actually uses (Flatten, not global pooling), climbs steadily with shift and is only a little better than raw pixels at large shifts: the CNN keeps position after Flatten, so it only gains robustness over a small range, not real invariance. This matches Task 1: the CNN kept 88.0% of its accuracy after a 1-pixel shift (FFNN 78.3%) and 62.0% after 2 pixels (FFNN 44.0%), but after 3 and 4 pixels both were about equal (27.2% against 26.2%, and 18.6% against 18.9%). We did not train a global-pooling version, so we cannot say how much it would improve this.

6 Where the symmetry stops: rotation and zoom

Convolution is built for translations only. For a rotation R (with inverse R^{-1} , since $(Rx)(u) = x(R^{-1}u)$) we have the following.

Lemma 6.1. $(Rx) \star (Rw) = R(x \star w)$. *Rotating the input gives the rotated output only if the kernel is rotated as well.*

Proof. **Step 1.** Expand the left side using the definition of convolution, with Rx as the image and Rw as the kernel:

$$((Rx) \star (Rw))(u) = \sum_v (Rx)(u+v) (Rw)(v).$$

Step 2. Expand both rotated terms using $(Rx)(p) = x(R^{-1}p)$:

$$(Rx)(u+v) = x(R^{-1}(u+v)), \quad (Rw)(v) = w(R^{-1}v).$$

Substituting,

$$((Rx) \star (Rw))(u) = \sum_v x(R^{-1}(u+v)) w(R^{-1}v).$$

Step 3. Use that R^{-1} , being a rotation matrix, distributes over addition:

$$R^{-1}(u+v) = R^{-1}u + R^{-1}v.$$

So

$$((Rx) \star (Rw))(u) = \sum_v x(R^{-1}u + R^{-1}v) w(R^{-1}v).$$

Step 4. Relabel the summation variable, $v' := R^{-1}v$ (equivalently $v = Rv'$). Because R is a rotation by a multiple of 90° , it permutes the finite set of kernel offsets v among themselves without loss or repetition, so summing over all v is the same as summing over all v' :

$$\sum_v x(R^{-1}u + v) w(v).$$

Step 5. Recognise this as a plain convolution, evaluated at $R^{-1}u$:

$$\sum_{v'} x(R^{-1}u + v') w(v') = (x \star w)(R^{-1}u).$$

Step 6. Recognise $(x \star w)(R^{-1}u)$ as a rotation of the function $x \star w$:

$$(x \star w)(R^{-1}u) = (R(x \star w))(u).$$

Chaining Steps 1–6 gives $((Rx) \star (Rw))(u) = (R(x \star w))(u)$ for every u , so $(Rx) \star (Rw) = R(x \star w)$. $\square$

Why Step 4 needs more care than the translation case. For shifts, relabelling the infinite grid $\mathbb{Z}^2$ by $v \mapsto v - s$ is automatically a bijection of the whole grid onto itself. For a finite kernel’s support (e.g. the 9 integer offsets of a 3×3 kernel), only rotations by multiples of 90° map those 9 integer offsets onto 9 other integer offsets exactly; a 45° rotation does not, and needs interpolation, which blurs the identity of Step 4 rather than preserving it exactly. This is a mathematical, not just practical, reason the exactness in this lemma is limited to 90° multiples on a square pixel grid.

The consequence for a trained CNN. The lemma says rotating the input matches rotating the output *only when the rotated kernel Rw is used*. A CNN has one fixed, learned kernel w ; rotating a test image never produces Rw automatically. So $(Rx) \star w$ — what actually happens when a trained kernel meets a rotated test image — is not covered by any equivariance guarantee at all; it is simply some other, unrelated feature map. In words, a vertical-edge detector on a rotated picture is a horizontal-edge detector on the original, but the network still applies the vertical-edge kernel, and mostly misses. We checked it with exact 90° rotations: with the same kernel, “rotate then convolve” differs from “convolve then rotate” by a relative error of 1.445; with the rotated kernel the error is 0 (Figure 3). A CNN has one set of kernels, so it does not treat a rotated object as the same object unless it learns the rotated kernels from data.

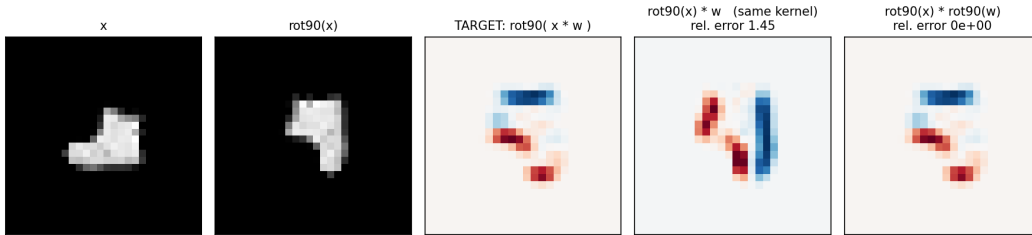


Figure 3: Rotation. The third panel is the target. Same kernel on the rotated image (fourth) is wrong; rotated kernel (fifth) is exact.

A square pixel grid only has exact rotations by multiples of 90° . Rotations like 45° or 60° need interpolation, which blurs the image, so no network can be exactly equivariant to them on pixels. Also, the symmetry should match the problem: rotating a “6” by 180° gives a “9”.

How to add rotation symmetry. One can train with rotated images (data augmentation), or use a group-equivariant CNN (Cohen and Welling, 2016), which applies the kernel in each of its four 90° rotations and combines the four resulting feature maps. We checked this construction: the equivariance relation it satisfies holds with error 0, and an invariant summary built from it changes by exactly 0 under a 90° rotation, against 24.1% for an ordinary convolution followed by a global sum. For a 45° rotation, which is not a symmetry of the pixel grid, it still changes by 11.7%, since no construction on a square grid is exactly equivariant to a non-multiple-of- 90° rotation. We did not train such a network ourselves.

Lemma 6.2 (Zoom). *For a zoom by factor a , $(D_a x) \star w = a^2 D_a(x \star w_a)$, where $w_a(v) = w(av)$ is the kernel rescaled by the same factor a .*

Proof. **Step 1.** Expand the left side using the definitions of zoom and convolution:

$$((D_a x) \star w)(u) = \sum_v (D_a x)(u+v) w(v) = \sum_v x\left(\frac{u+v}{a}\right) w(v).$$

Step 2. Substitute $v' = v/a$ (so $v = av'$). This is a change of variables on a continuous grid, which introduces a Jacobian factor of a^2 in two dimensions:

$$\sum_v x\left(\frac{u+v}{a}\right) w(v) = a^2 \sum_{v'} x\left(\frac{u}{a} + v'\right) w(av').$$

Step 3. Recognise $w(av')$ as $w_a(v')$ by definition, and recognise the remaining sum as a plain convolution of x with w_a , evaluated at u/a :

$$a^2 \sum_{v'} x\left(\frac{u}{a} + v'\right) w_a(v') = a^2 (x \star w_a)\left(\frac{u}{a}\right).$$

Step 4. Recognise $(x \star w_a)(u/a)$ as D_a applied to the function $x \star w_a$, by the definition of D_a :

$$(x \star w_a)\left(\frac{u}{a}\right) = (D_a(x \star w_a))(u).$$

Chaining Steps 1–4: $((D_a x) \star w)(u) = a^2 (D_a(x \star w_a))(u)$ for every u , so $(D_a x) \star w = a^2 D_a(x \star w_a)$. $\square$

The same structural message as rotation. Zooming the input matches zooming the output only if the kernel is rescaled too, w_a rather than w : a fixed 3×3 kernel is tied to one specific spatial scale and cannot automatically stretch to match a zoomed-in or zoomed-out version of the same pattern. This is why rotation and zoom are treated together in this report: both are symmetries a plain, fixed convolution structurally lacks, for the same underlying reason.

Task 1 results. Table 3 shows the accuracy kept (as a percentage of the accuracy on the original test images). Both networks were trained only on original images.

The CNN is better for small rotations, small shifts and zooms, but neither network is invariant. At 60° and 90° both are below the 10% chance level of a 10-class problem, which agrees with Lemma 6.1: nothing in the architecture handles rotation. Zoom shows a further, asymmetric pattern: shrinking the object (zoom 0.7) is punished more heavily than enlarging it (zoom 1.15), likely because shrinking also shrinks the object’s edges relative to the kernel’s fixed window, pushing features below the kernel’s effective resolution; the drop-off at zoom 1.3 has a second, separate cause, the fixed 28×28 canvas simply cropping parts of the enlarged object, which is a canvas artifact rather than a symmetry failure.

Table 3: Accuracy kept after transforming the test images (our Task 1 run).

Transformation	FFNN	CNN
rotate 15°	62.7%	77.2%
rotate 30°	23.6%	30.0%
rotate 45°	13.0%	13.5%
rotate 60°	8.1%	7.8%
rotate 90°	5.5%	3.1%
shift 1 px	78.3%	88.0%
shift 2 px	44.0%	62.0%
shift 3 px	26.2%	27.2%
shift 4 px	18.9%	18.6%
zoom 0.7	40.0%	66.0%
zoom 0.85	71.7%	90.6%
zoom 1.15	83.8%	90.4%
zoom 1.3	69.8%	79.4%

7 Scale separation

Symmetry (Sections 4–6) explains *what* a convolution layer does not change (or does change predictably) when the input moves. It says nothing yet about *scale*: images contain structure at many sizes at once — a boot has fine edges, medium-sized parts (the sole, the ankle), and an overall large-scale silhouette. This section asks how a network should be built to see all of these sizes efficiently, and answers it by tracing the same question through two classical tools of signal processing, the Fourier transform and the wavelet transform, before connecting the answer to the actual layers of our CNN.

7.1 The Fourier transform, and what it cannot tell you

The Fourier transform represents a signal as a sum of sine waves of different frequencies:

$$\hat{x}(\omega) = \int x(u) e^{-i\omega u} du. \quad (4)$$

Every frequency ω gets one complex number $\hat{x}(\omega)$, describing how strongly that frequency is present in x and with what phase. This representation is extremely good at answering “what frequencies (how fine or coarse a pattern) are present in the signal” — but, as the next theorem shows precisely, it is completely blind to *where* in the signal those frequencies occur.

Theorem 7.1 (Fourier shift theorem). $\widehat{T_s x}(\omega) = e^{-i\omega s} \hat{x}(\omega)$.

Proof. **Step 1.** Expand the left side using the definition of the Fourier transform, applied to the shifted signal $T_s x$:

$$\widehat{T_s x}(\omega) = \int (T_s x)(u) e^{-i\omega u} du = \int x(u - s) e^{-i\omega u} du.$$

Step 2. Substitute $u' = u - s$ (so $u = u' + s$, $du = du'$):

$$\int x(u - s) e^{-i\omega u} du = \int x(u') e^{-i\omega(u'+s)} du'.$$

Step 3. Split the exponent, since $e^{-i\omega(u'+s)} = e^{-i\omega u'} e^{-i\omega s}$, and pull the constant factor $e^{-i\omega s}$ out of the integral:

$$\int x(u') e^{-i\omega u'} e^{-i\omega s} du' = e^{-i\omega s} \int x(u') e^{-i\omega u'} du'.$$

Step 4. Recognise the remaining integral as $\hat{x}(\omega)$, by the definition of the Fourier transform:

$$e^{-i\omega s} \int x(u') e^{-i\omega u'} du' = e^{-i\omega s} \hat{x}(\omega).$$

Chaining Steps 1–4: $\widehat{T_s x}(\omega) = e^{-i\omega s} \hat{x}(\omega)$. □

What this means for position information. The factor $e^{-i\omega s}$ has magnitude 1 for every real ω and s (it is a point on the unit circle), so

$$|\widehat{T_s x}(\omega)| = |e^{-i\omega s}| \cdot |\hat{x}(\omega)| = |\hat{x}(\omega)|.$$

The magnitude spectrum $|\hat{x}(\omega)|$ is therefore *exactly* shift-invariant — shifting the input changes it not at all. This sounds desirable, but it is invariance in the wrong place and to an unusable degree: all information about *where* a feature sits is pushed entirely into the phase term $e^{-i\omega s}$, and phase is difficult to read off directly frequency by frequency. A single sine wave, the basic building block of the Fourier representation, extends over the *entire* signal; knowing that a particular frequency is present tells you that some feature of that scale exists somewhere in the image, but not where. This is the opposite of what a first layer of a vision network needs: Section 3 argued that early layers should be *equivariant* (moving with the object, so that position is preserved and useable), and pure Fourier magnitude is invariant in a way that discards position entirely. Fourier analysis is excellent at separating an image by frequency; it is unusable, by itself, for saying where in the image each frequency lives.

7.2 The wavelet transform: localising both position and scale

The wavelet transform fixes exactly this gap. Instead of infinitely long sine waves, it uses a small, localised, oscillating function ψ (a **mother wavelet**, with zero average so it only responds to change, not to a constant background), and analyses the signal with dilated copies of it:

$$\psi_j(u) = 2^{-j} \psi(u/2^j), \quad (x \star \psi_j)(u) = \text{the wavelet coefficient of } x \text{ at scale } j, \text{ position } u. \quad (5)$$

Small j gives a narrow, compressed wavelet, sensitive to fine detail (edges, texture); large j gives a wide, stretched wavelet, sensitive to coarse, large-scale structure. Crucially, $x \star \psi_j$ is — by its very definition — an ordinary convolution, with a fixed kernel ψ_j . Theorem 4.1 therefore applies to it directly: each wavelet scale, on its own, is exactly translation equivariant, so the wavelet transform keeps position information (unlike the raw Fourier magnitude) while still separating the signal by scale (like Fourier). Computing $x \star \psi_j$ for a whole ladder of scales $j = 0, 1, 2, \dots$ gives a **multi-resolution decomposition**: the same image, described simultaneously at many sizes, each description still knowing where its features are.

This is scale separation, defined precisely: rather than a single global analysis (Fourier) or a single fixed-size local analysis (one convolution layer with one kernel size), a multi-resolution decomposition looks at the image through a whole family of localised probes at growing scales, $2^0, 2^1, 2^2, \dots$, each one a convolution and each therefore individually translation equivariant.

7.3 From wavelets to CNNs: a learned, cascaded multi-resolution pyramid

A classical wavelet transform fixes the mother wavelet ψ in advance (a mathematician chooses its shape) and computes all scales j directly from the raw input x . A CNN differs in two ways, both of which make it more flexible, at the cost of needing training data.

First, a CNN does *not* fix the filter shape by hand. The first convolution layer learns its own small kernels from data, effectively learning whichever fine-scale filters (edge detectors at various orientations, colour-opponent filters) turn out to be useful for the task, rather than committing in advance to one hand-designed ψ .

Second, and more importantly for scale separation, a CNN does not compute every scale directly from the raw pixels the way a classical wavelet transform does. Instead it *cascades*: pooling or striding shrinks the current feature map by roughly a factor of 2 (Section 5), and the *next* convolution layer is then applied to this already-coarsened map. A kernel of the same fixed size, say 3×3 , applied after one 2×2 pooling step, is looking at a 3×3 window of a once-halved image — which corresponds to roughly a 6×6 window of the original, uncoarsened image. Repeating convolution-then-pooling several times reproduces, layer by layer, exactly the dyadic (doubling) ladder of scales $2^0, 2^1, 2^2, \dots$ that a wavelet transform builds directly: each pooling stage is one step to the next, coarser octave, and the convolution that follows it is a learned filter operating natively at that octave’s resolution, rather than a fixed-size filter having to reach further and further across the original image.

Table 4 makes this correspondence concrete, layer by layer, for our own implemented CNN.

Table 4: Our CNN as a cascaded, learned, coarse-to-fine pyramid.

Layer	Operation	Grid size	Role in the pyramid
input	raw image	28×28	finest scale; no filtering yet
conv1 + ReLU	learned 3×3 kernels	26×26	fine-scale, equivariant filters (edges), analogous to the smallest-scale wavelets ψ_0
pool1	2×2 max, stride 2	13×13	one step to the next octave: halves resolution, roughly doubles each unit’s effective coverage of the original image
conv2 + ReLU	learned 3×3 kernels	11×11	medium-scale filters, now combining the already-local features from conv1 into larger patterns (corners, textures), analogous to a coarser wavelet ψ_1 acting on the scale-0 coefficients
pool2	2×2 max, stride 2	5×5	a second step to a coarser octave
Flatten	+ fully connected	$400 \rightarrow 64 \rightarrow$	combines the coarsest-scale features into a class decision; position information kept, equivariance stops here (see below)
Dense		10	

Reading the table top to bottom shows scale separation as a construction with two moving parts, applied in alternation: a convolution (a local, equivariant filter, exactly as proved in Theorem 4.1) followed by a coarsening step (pooling, exactly as proved in Theorem 5.1), repeated to build progressively coarser octaves. The two theorems already proved in this report are therefore not just facts about isolated layers — chained together, they are precisely the construction that produces a

multi-resolution, wavelet-like pyramid, except with a learned rather than a hand-designed filter at every scale.

Where our CNN departs from a purely wavelet-like construction. A pure wavelet cascade could end with a global summary at the coarsest scale (Section 5 showed this gives exact shift invariance). Our CNN instead ends with **Flatten**, which relabels the $5 \times 5 \times 16$ feature map as a 400-number list, and this step is equivariant to nothing at all: two feature-map positions that were still meaningfully related through every convolution and pooling step before it are now sent to two unrelated coordinates of a flat vector, and the dense layers that follow are free — and, empirically, do — treat them completely differently. This is a deliberate design choice, shared with classic architectures such as LeNet-5 (LeCun et al., 1998): it trades some of the robustness a fully scale-and-position-separated pyramid would have for the ability to use absolute position when it happens to be informative (for example, if the object of interest tends to sit in a particular part of the frame). It is not a flaw in the implementation; it is where, precisely, the chain of equivariant reasoning built up over Sections 4–5 stops.

7.4 Receptive field: the concrete numbers

The **receptive field** of a unit is the set of input pixels that can change it. If r_l is its size after layer l and j_l is the distance, in input pixels, between neighbouring units, then for a layer with kernel size k_l and stride s_l :

$$r_l = r_{l-1} + (k_l - 1)j_{l-1}, \quad j_l = j_{l-1}s_l. \quad (6)$$

The reasoning is direct: a unit looks at k_l neighbouring units of the previous layer, and those neighbouring units are j_{l-1} pixels apart in the original image, so the extra pixels reached span $(k_l - 1)j_{l-1}$; a stride multiplies the spacing between units for every later layer, which is why j_l grows multiplicatively rather than additively. This recursion is exactly the pixel-counting version of Section 7.3’s octave ladder: each pooling stage (a stride-2 layer) doubles j_l , matching a wavelet transform’s dyadic scale 2^j , and each convolution widens r_l by its kernel size at whatever the current scale already is.

For our CNN this gives Table 5.

Table 5: Receptive field of one unit in our CNN.

Layer	Kernel	Stride	Receptive field	Jump
conv1 3×3	3	1	3×3	1
pool1 2×2	2	2	4×4	2
conv2 3×3	3	1	8×8	2
pool2 2×2	2	2	10×10	4
Dense (after flatten)	—	—	28×28 (all)	—

We also measured it directly: we increased each input pixel of a random conv-pool-conv-pool network in turn and recorded which pixels changed one particular output unit. The result is a 10×10 block of 100 pixels, exactly as the formula predicts (Figure 4). Without pooling, the receptive field grows only by 2 per 3×3 layer (eight such layers would see just 17 pixels); with pooling after each convolution it roughly doubles at each stage (3, 4, 8, 10, 18, 22, 38, 46 for eight layers), so it covers the whole 28-pixel image after about seven layers. Small kernels are therefore enough to eventually see a whole image, precisely because pooling advances the octave ladder geometrically rather than the kernel having to grow to cover more pixels directly.

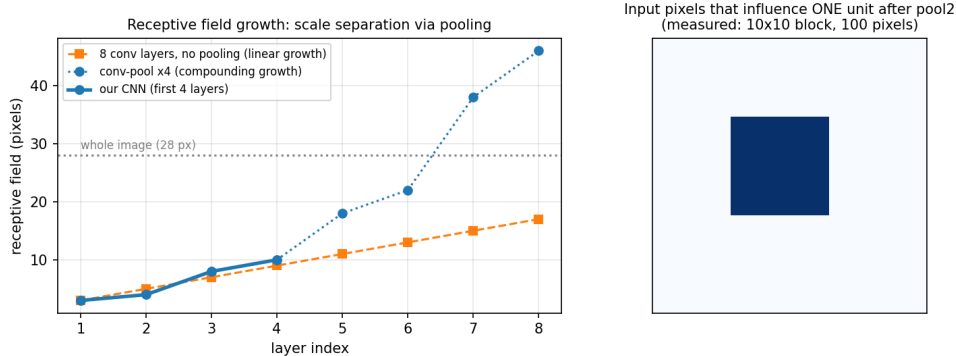


Figure 4: Left: receptive field growth with and without pooling. Right: the measured 10×10 receptive field of one unit after the second pooling.

7.5 Cost, approximation and stability

Cost. Scale separation and weight sharing save parameters, but not operations. Each weight is reused at many positions, so the CNN needs 443,360 floating-point operations per image against 218,560 for the FFNN, and about 9 times more activation memory in training (4,440 KB against 490 KB). This explains why the CNN was slower in Task 1.

Why the hierarchy helps learning. Poggio et al. (2017) show that for functions built by composing simple local functions — exactly the compositional structure a wavelet-like, layer-by-layer pyramid assumes about images — deep networks that follow this structure need a number of parameters that depends only on the small local input size (here, a 3×3 patch) and not on the full input dimension (784). We did not test this theorem ourselves; it only explains the direction of our results.

Stability to small deformations. The wavelet cascade of Section 7.3 is not just an analogy: Mallat (2012) constructed the **scattering transform**, a mathematically rigorous network built from exactly this recipe — a cascade of (fixed, hand-designed) wavelet convolutions, a modulus nonlinearity, and a final averaging step (a global coarsening, in the sense of Section 5) — and proved that the resulting representation is close to translation invariant up to the averaging scale, and changes only a little under small deformations of the image. A rotation by angle θ is a deformation of size about $2 \sin(\theta/2)$, which is 0.26 for 15° and 1.0 for 60° ; a zoom by a has size $|1 - 1/a|$, for example 0.13 for $1.15\times$. Small deformations are tolerated and large ones are not, which is what Table 3 shows (15° rotation and $1.15\times$ zoom are tolerated, 60° is not). This is only a rough trend, and the scattering-transform theorem was not independently reproven here; it is included because it is the closest thing in the literature to a formal proof of why a wavelet-like, multi-scale cascade — the same shape as our CNN’s conv-pool-conv-pool stack — should be expected to behave stably under exactly the transformations Table 3 tests.

Feature hierarchy. The file `results/feature_hierarchy.png` from our run shows the learned features directly. The CNN’s first-layer kernels look like small edge detectors, matching the finest-scale wavelets ψ_0 of Section 7.2; the second-layer maps respond to larger structures (an 8×8 effective window, per Table 5), matching a coarser scale ψ_1 ; the FFNN’s first-layer weights, by contrast, look like unstructured noise, because each of its units is connected to the whole image and nothing in its construction forces locality or a preferred scale.

8 Link with the Task 1 results

- **Fewer parameters (27,562 against 109,386):** locality and weight sharing (Section 4).
- **Same accuracy on centred images:** when objects are always at the same place, there is nothing for the symmetry to help with, and the FFNN can learn absolute positions.
- **Better when images are randomly shifted by up to 3 pixels (77.3% against 74.6%):** now equivariance and local invariance matter. This is from one run, so it shows the direction and not an exact gap.
- **Smaller train-test gap (5.7 against 7.2 points):** a smaller hypothesis class; each image acts like many shifted images.
- **Advantage only for shifts of 1 or 2 pixels:** pooling invariance is local (Section 5) and the flatten layer keeps position.
- **Both fail at 60° and 90°:** convolution is not rotation equivariant (Lemma 6.1).
- **CNN slower and heavier:** weight sharing saves parameters, not operations; our NumPy implementation is also not as optimised as GPU libraries.

9 Limitations

- The Task 1 numbers come from one training run on 10,000 training images with small models. Differences of about one point or less are within noise.
- The numerical checks use one image and fixed kernels. The theorems are general; the numbers only confirm them.
- Rotated and zoomed images were made with bilinear interpolation, which blurs them, so part of the accuracy loss comes from the interpolation.
- The results of [Poggio et al. \(2017\)](#) and [Mallat \(2012\)](#) are quoted in informal form. We did not train a global-pooling CNN or a rotation-equivariant CNN; they are natural next experiments.

10 Conclusion

The CNN works well for images because its structure matches two properties of images. Convolution is exactly the linear operation that respects translation symmetry, so one small kernel can be reused at every position, which reduces the parameters and the overfitting. Pooling turns this equivariance into (local) invariance, and stacking convolution and pooling builds, layer by layer, the same kind of multi-resolution, coarse-to-fine pyramid that the Fourier and wavelet transforms were built to produce for one-dimensional signals — with the crucial improvement that the filters at each scale are learned rather than fixed. The same mathematics shows the limits: rotation and scale are not symmetries of the layers, and a flatten layer keeps position, so the invariance is only over a small range. The Task 1 results agree with this: fewer parameters, less overfitting, better robustness to small transformations, the same accuracy on centred data, higher computation cost, and failure under large rotations.

References

- Cohen, T., and Welling, M. (2016). Group Equivariant Convolutional Networks. *ICML*.
- LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- Mallat, S. (1999). *A Wavelet Tour of Signal Processing*. Academic Press.
- Mallat, S. (2012). Group Invariant Scattering. *Communications on Pure and Applied Mathematics*, 65(10).
- Poggio, T., Mhaskar, H., Rosasco, L., Miranda, B., and Liao, Q. (2017). Why and When Can Deep, but Not Shallow, Networks Avoid the Curse of Dimensionality: A Review. *International Journal of Automation and Computing*, 14.
- Xiao, H., Rasul, K., and Vollgraf, R. (2017). Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms. *arXiv:1708.07747*.
- Zhang, R. (2019). Making Convolutional Networks Shift-Invariant Again. *ICML*.